

Section 11.2 - Preparing text data

December 17, 2025

1 Preparing text data

- Text standardization
- Tokenization
- Vocabulary indexing
- The TextVectorization layer

1.1 Text standardization

- Similar sentences have similar representations - less training data required & generalize better
- Keep the context in mind
- Most widespread scheme:
 - 1) convert to lowercase
 - 2) remove punctuation characters
 - 3) convert special characters to standard forms
 - 4) word stemming (rarely used)

1.2 Tokenization

- Break text into units/tokens
- Word-level tokenization (subword, character-level?)
- N-grams tokenization

1.3 Vocabulary indexing

- Encode tokens to numerical representations
- Build an index (vocabulary) of terms in training data
- Assign a unique integer to each token
- out-of-vocabulary (OOV) token = 1, decode[1] = [UNK]
- mask token = 0 (not a word) to pad sequences to create batches of same length samples

1.4 The TextVectorization layer

- output_mode = “int”, “multi_hot”, “count”, “tf-idf”
- Use custom functions for standardization and tokenization
- Two ways of usage:
 - 1) In the tf.data pipeline - better performance on GPU

2) As part of the model - better for production environment

→ Overall, Train with tf.data pipeline, and export the trained model incorporating text processing.

```
[1]: # TextVectorization

import string

class Vectorizer:
    def standardize(self, text):
        text = text.lower()
        return "".join(char for char in text if char not in string.punctuation)

    def tokenize(self, text):
        text = self.standardize(text)
        return text.split()

    def make_vocabulary(self, dataset):
        self.vocabulary = {"": 0, "[UNK]": 1}
        for text in dataset:
            text = self.standardize(text)
            tokens = self.tokenize(text)
            for token in tokens:
                if token not in self.vocabulary:
                    self.vocabulary[token] = len(self.vocabulary)
        self.inverse_vocabulary = dict(
            (v, k) for k, v in self.vocabulary.items())

    def encode(self, text):
        text = self.standardize(text)
        tokens = self.tokenize(text)
        return [self.vocabulary.get(token, 1) for token in tokens]

    def decode(self, int_sequence):
        return " ".join(
            self.inverse_vocabulary.get(i, "[UNK]") for i in int_sequence)

vectorizer = Vectorizer()
dataset = [
    "I write, erase, rewrite",
    "Erase again, and then",
    "A poppy blooms.",
]
vectorizer.make_vocabulary(dataset)
```

Encode a text sequence

```
[2]: test_sentence = "I write, rewrite, and still rewrite again"
      encoded_sentence = vectorizer.encode(test_sentence)
      print(encoded_sentence)
```

[2, 3, 5, 7, 1, 5, 6]

Decode an integer sequence

```
[3]: decoded_sentence = vectorizer.decode(encoded_sentence)
      print(decoded_sentence)
```

i write rewrite and [UNK] rewrite again

The TextVectorization in Keras

```
[4]: from tensorflow.keras.layers import TextVectorization
      text_vectorization = TextVectorization(
          output_mode="int",
      )
```

Custom standardization and tokenization

```
[5]: import re
      import string
      import tensorflow as tf

      def custom_standardization_fn(string_tensor):
          lowercase_string = tf.strings.lower(string_tensor)
          return tf.strings.regex_replace(
              lowercase_string, f"[{re.escape(string.punctuation)}]", "")

      def custom_split_fn(string_tensor):
          return tf.strings.split(string_tensor)

      text_vectorization = TextVectorization(
          output_mode="int",
          standardize=custom_standardization_fn,
          split=custom_split_fn,
      )
```

```
[6]: dataset = [
      "I write, erase, rewrite",
      "Erase again, and then",
      "A poppy blooms.",
      ]
      text_vectorization.adapt(dataset)
```

Displaying the vocabulary

```
[7]: text_vectorization.get_vocabulary()
```

```
[7]: ['',
      '[UNK]',
      'erase',
      'write',
      'then',
      'rewrite',
      'poppy',
      'i',
      'blooms',
      'and',
      'again',
      'a']
```

Encode a text sequence

```
[8]: vocabulary = text_vectorization.get_vocabulary()
test_sentence = "I write, rewrite, and still rewrite again"
encoded_sentence = text_vectorization(test_sentence)
print(encoded_sentence)
```

```
tf.Tensor([ 7  3  5  9  1  5 10], shape=(7,), dtype=int64)
```

Decode an integer sequence

```
[9]: inverse_vocab = dict(enumerate(vocabulary))
decoded_sentence = " ".join(inverse_vocab[int(i)] for i in encoded_sentence)
print(decoded_sentence)
```

```
i write rewrite and [UNK] rewrite again
```